

JavaScript Promises

A QUICK REFERENCE

The Most Complete Full Stack
Developer Roadmap

www.thedevspace.io

A Promise represents a value that may be available in the future, allowing the program to handle something else while waiting for the promise to settle.

CREATING PROMISE

Create promises with the **Promise** constructor or helpers like **Promise.resolve()/Promise.reject()**.



```
1 const p1 = new Promise((res) => setTimeout(() => res(1), 100));  
2 const p2 = Promise.resolve(2);
```

RESOLVING PROMISE

Resolve a promise with a value or another Promise, or reject with an error.

```
1  const resolved = new Promise((resolve, reject) => {
2    const ok = true; // flip to false to see the reject path
3    if (ok) resolve(3);
4    else reject(new Error("nope"));
5  });
6
7  resolved
8    .then((v) => console.log(v)) // 3 when ok === true
9    .catch((e) => console.error(e));
```

PROMISE CHAINING

- Chain multiple **.then()** to sequence async steps
- **.catch()** handles errors
- **.finally()** always runs

```
1 fetch("/api")
2   .then((r) => r.json())
3   .then((data) => save(data))
4   .catch((err) => handle(err))
5   .finally(() => cleanup());
```

ASYNC / AWAIT

Use **async/await** syntax for readable sequential async code.



```
1  async function load() {  
2    try {  
3      const res = await fetch("/api");  
4      return await res.json();  
5    } catch (err) {  
6      console.error(err);  
7      throw err;  
8    }  
9  }
```

PROMISE.ALL()

Waits for all **Promises** to fulfill. Rejects immediately on the first rejection.

```
1 Promise.all([fetch(a), fetch(b)])  
2   .then(([ra, rb]) => {  
3     /* both succeeded */  
4   })  
5   .catch((err) => {  
6     /* one failed */  
7   });
```


PROMISE.RACE()

Resolves or rejects as soon as the first **Promise** settles.



```
1 Promise.race([p1, p2]).then((first) => console.log(first));
```


PROMISE.ALLSETTLED()

Waits for all **Promises** to settle and returns their statuses (useful when you need results regardless of failures).



```
1 const results = await Promise.allSettled([p1, p2]);  
2 // inspect results: { status: 'fulfilled'|'rejected', value|reason }
```

PROMISE.ANY()

Resolves with the first fulfilled **Promise**, rejects only if all input Promises reject.



```
1 Promise.any([p1, p2])
2   .then((value) => console.log("fastest success", value))
3   .catch((allRejected) => console.error("all failed"));
```